

S C S 1 3 0 3

Introduction to Software Engineering

Section 3 · Agile Software Development

Prof. K. P. Hewagamage

B.Sc. Computer Science · Year 1, Semester 1
University of Colombo School of Computing

Section 3 — Intended Learning Outcomes

By the end of this section, students should be able to:

- LO1 Explain the Agile Manifesto and its 12 principles in plain language
- LO2 Describe Scrum — roles, events, artifacts, and the sprint cycle
- LO3 Explain XP's key practices — TDD, pair programming, and small releases
- LO4 Describe how a Kanban board works and what WIP limits achieve
- LO5 Explain Lean's origins and its seven principles
- LO6 Choose an appropriate agile approach for a given project context
- LO7 Identify the main agile tools (Jira, Trello) and how they support teams
- LO8 Describe how AI is beginning to change agile software development practices

Section 3 — Roadmap

Six topics across 4 lecture hours — building from foundations to modern practice

3.1

Agile culture — rapid recap

3.2

Scrum — the dominant framework

3.3

Extreme Programming (XP)

3.4

Kanban — visualise and flow

3.5

Lean and other agile methods

3.6

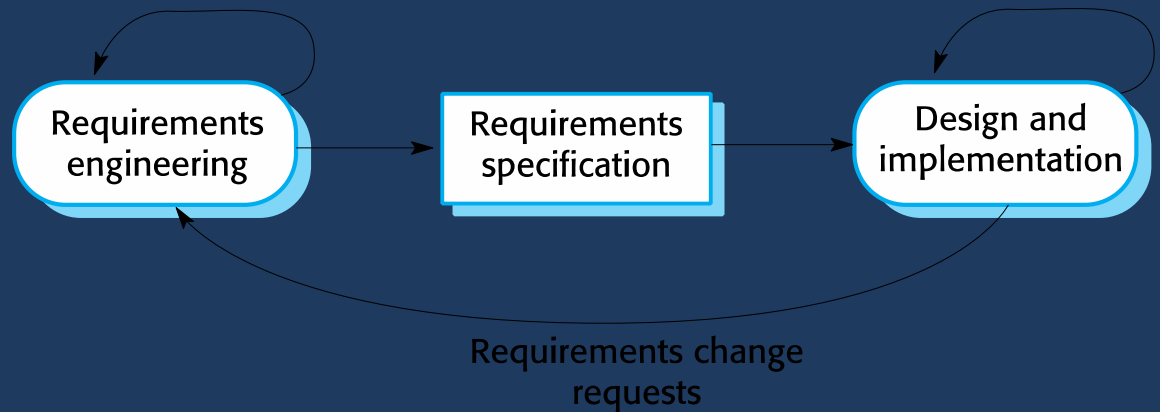
Tools and AI in Agile

3.1

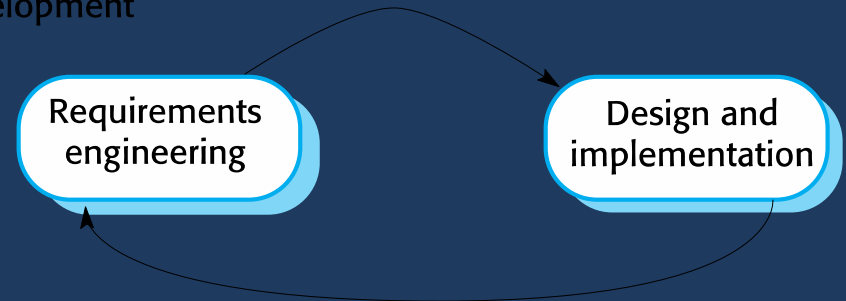
Agile Culture

From plan-driven to AgileHeading towards

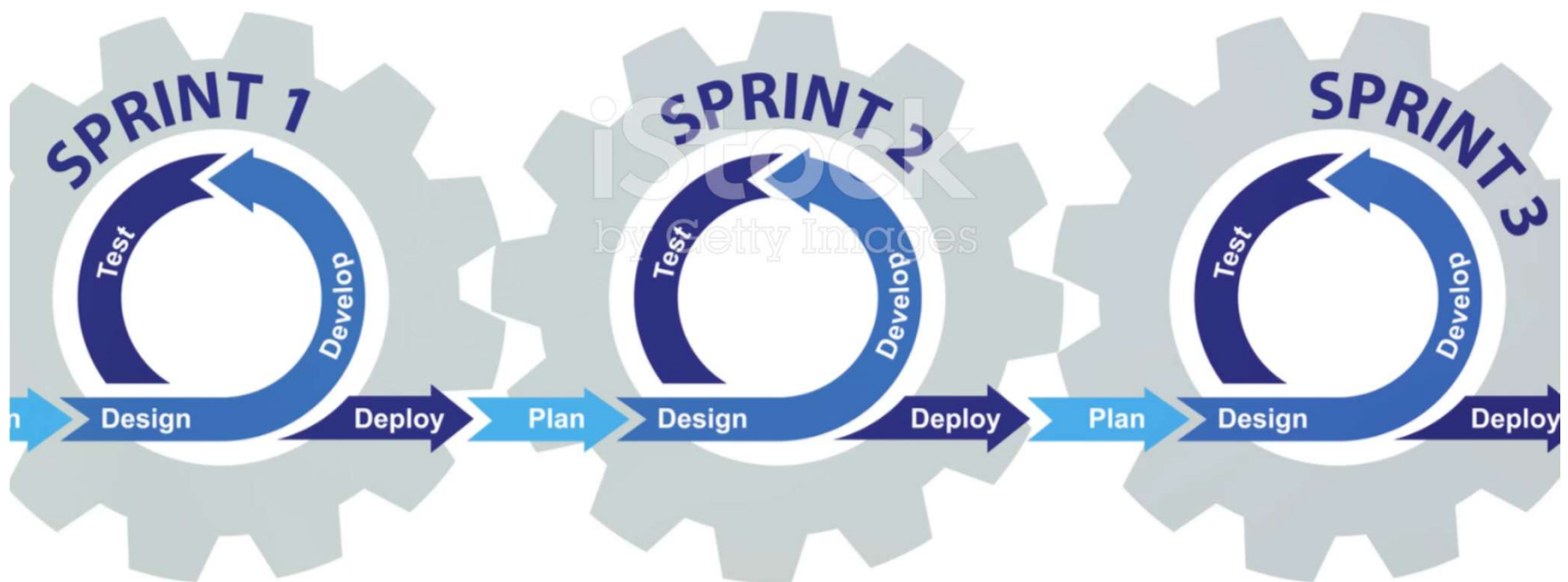
Plan-based development



Agile development



AGILE METHODOLOGY



Why Agile? The Case for Change

Key insight

Plan-driven approaches assume you can fully define requirements upfront. In most modern software projects, **that assumption is wrong.**

Requirements change constantly

Business needs shift, markets evolve, technology advances — *plans made in January are wrong by April*

Customers can't specify upfront

Users don't know exactly what they want until they can see and touch a working system

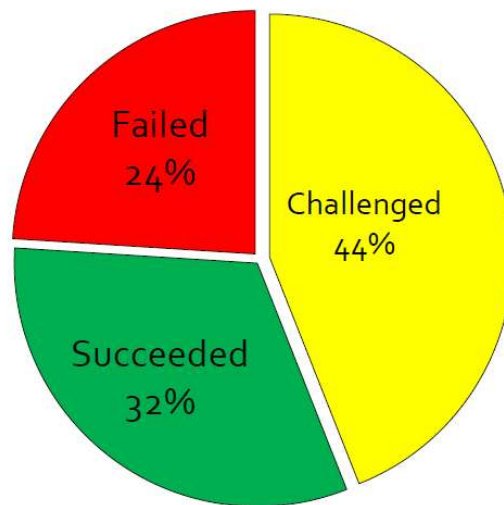
Speed to market is critical

In a competitive environment, releasing working software in weeks beats planning for a year

Teams need flexibility

Rigid processes create bottlenecks; empowered, self-organising teams respond faster

Status of Software Projects in 10 Years ago...

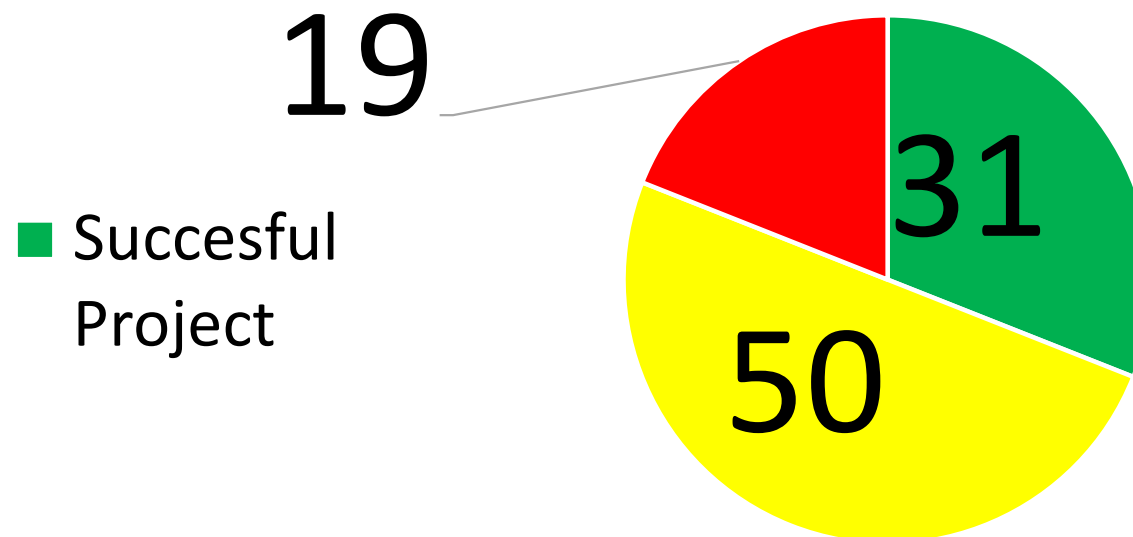


- 32% of all projects succeeded (on time, on budget, with required features)
- 44% are challenged (late, over budget and/or with less than the required features)
- 24% have failed (cancelled prior to completion and/or never used)

Source: Standish CHAOS Report 2009
Boston, Massachusetts, April 23, 2009

Status of Software Projects after 10 Years

Sandish CHOS Report 2024



The Agile Manifesto — Four Values

Snowbird, Utah, 2001 · 17 practitioners · one shared statement

The Manifesto does not say the items on the right have no value. It says we value the items on the left MORE.

Individuals and interactions

over

processes and tools

Working software

over

comprehensive documentation

Customer collaboration

over

contract negotiation

Responding to change

over

following a plan

The Agile Manifesto — 12 Principles (1 of 2)

- | | | |
|---|------------------------------------|--|
| 1 | Satisfy the customer | Highest priority is early and continuous delivery of valuable software |
| 2 | Welcome change | Even late in development — use it for the customer's competitive advantage |
| 3 | Deliver frequently | Working software every few weeks; the shorter the timescale, the better |
| 4 | Work together daily | Business people and developers must collaborate throughout the project |
| 5 | Trust motivated individuals | Give them environment and support; trust them to get the job done |
| 6 | Face-to-face conversation | The most effective method of conveying information within a team |

The Agile Manifesto — 12 Principles (2 of 2)

7

Working software = progress

Working software is the primary measure of progress — not documents

8

Sustainable pace

Sponsors, developers, users should maintain a constant pace indefinitely

9

Technical excellence

Continuous attention to good design and engineering enhances agility

10

Simplicity

Maximise the amount of work NOT done — do only what is needed

11

Self-organising teams

The best architectures and designs emerge from self-organising teams

12

Reflect and adapt

Regularly reflect on how to be more effective and adjust accordingly

The Agile Methods Family

All are built on the Manifesto values — each emphasises something different

Scrum

Framework · Roles ·
Sprints

XP

Engineering excellence

Kanban

Visualise · Flow · WIP
limits

Lean

Eliminate waste · Deliver
fast

Crystal

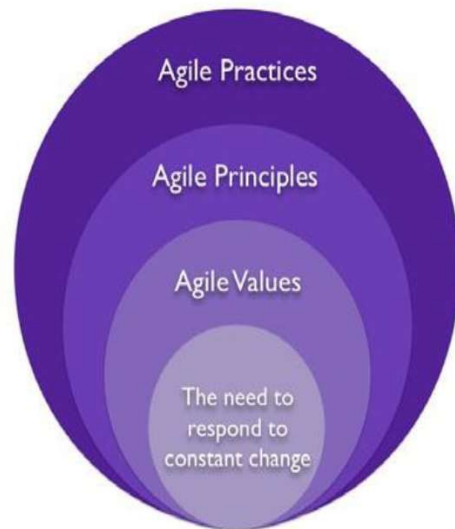
People · Communication ·
Safety

FDD

Feature-by-feature
delivery

We cover Scrum, XP, Kanban, and Lean in depth. Crystal and FDD at overview level.

Relationship between Agile Values, Principles and Practices



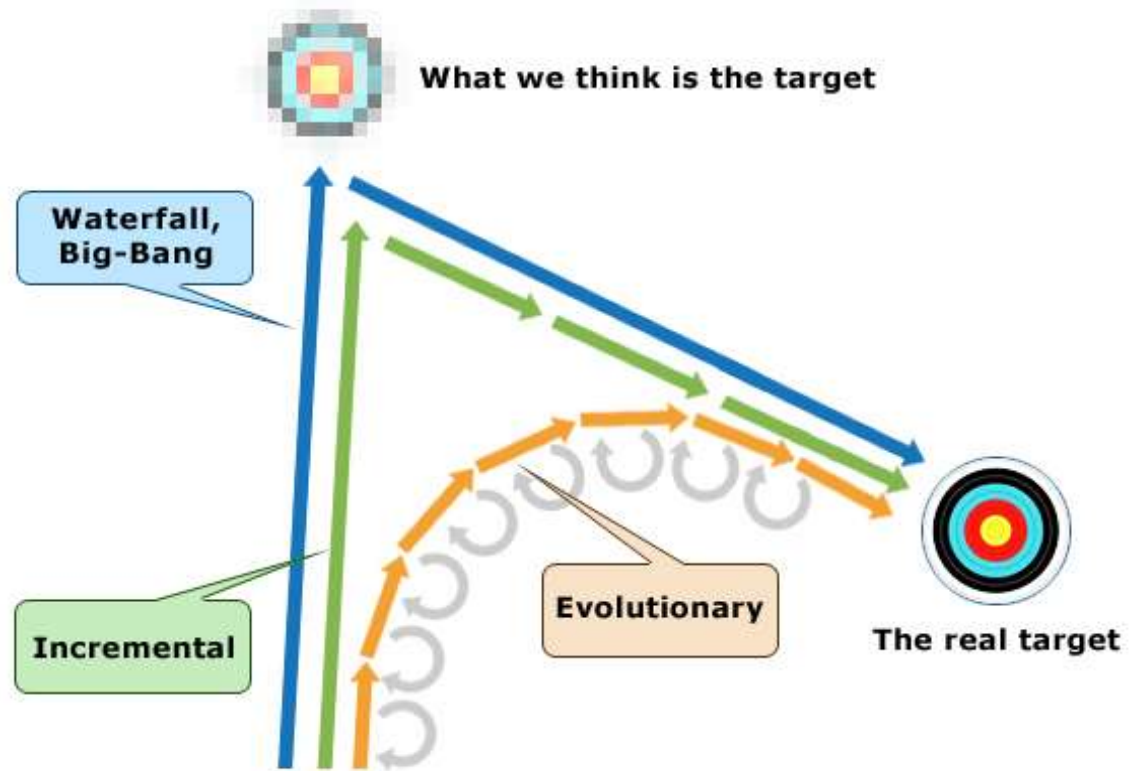
- 4 Values
- 12 Principles
- Many practices, grouped with as methodologies
 - E.g. XP, SCRUM,

Agile Software Development:

Learn → Adapt → Deliver

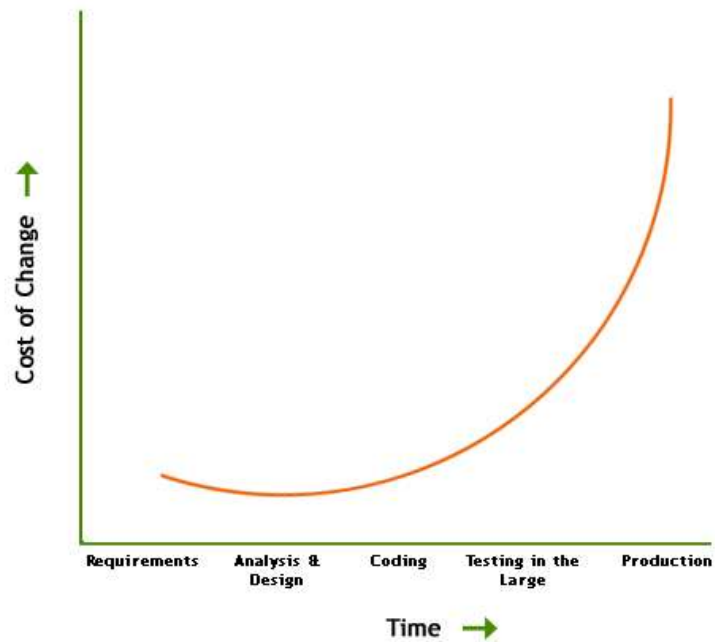
- Ability to adapt
 - To changing requirements, To changing circumstances
- Agile Software development?
 - based on Learn & Adapt rather than rigid processes
 - Evolving systems in short iterations
 - Each release is a working system
 - Focus on business value
 - Design for change
 - Communicating efficiently & real time visibility to progress
 - Leveraging human strengths
 - Engage, align, and empower the team
 - Get power from each member
- Agile is not an excuse for “no discipline”

Reaching the Goal

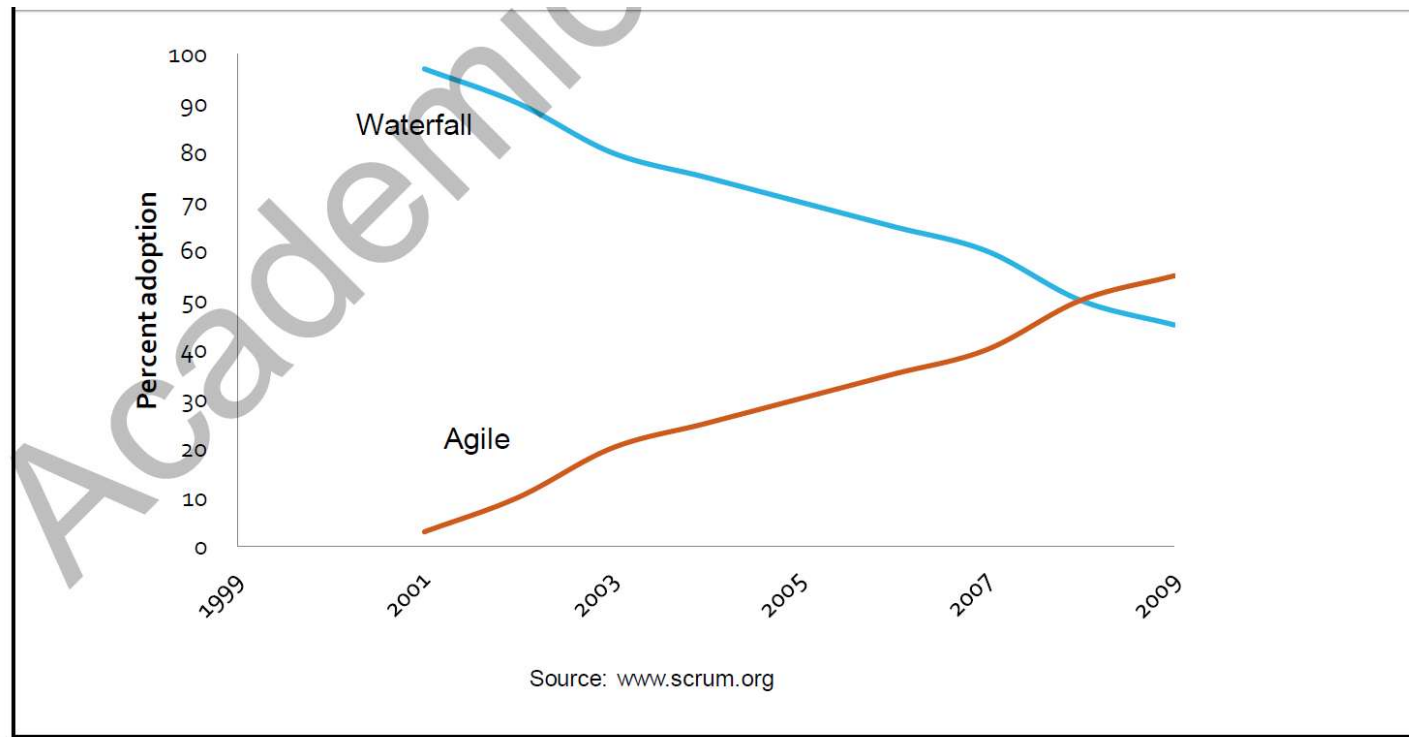


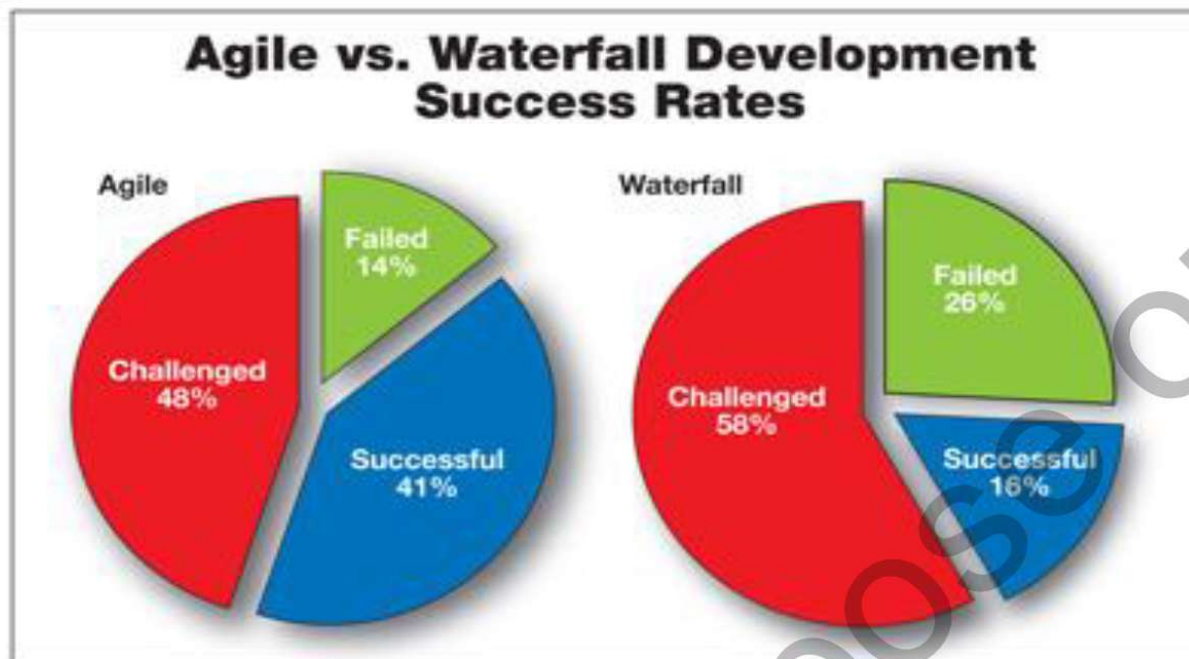
Why Agile?

COST OF CHANGE - WATERFALL



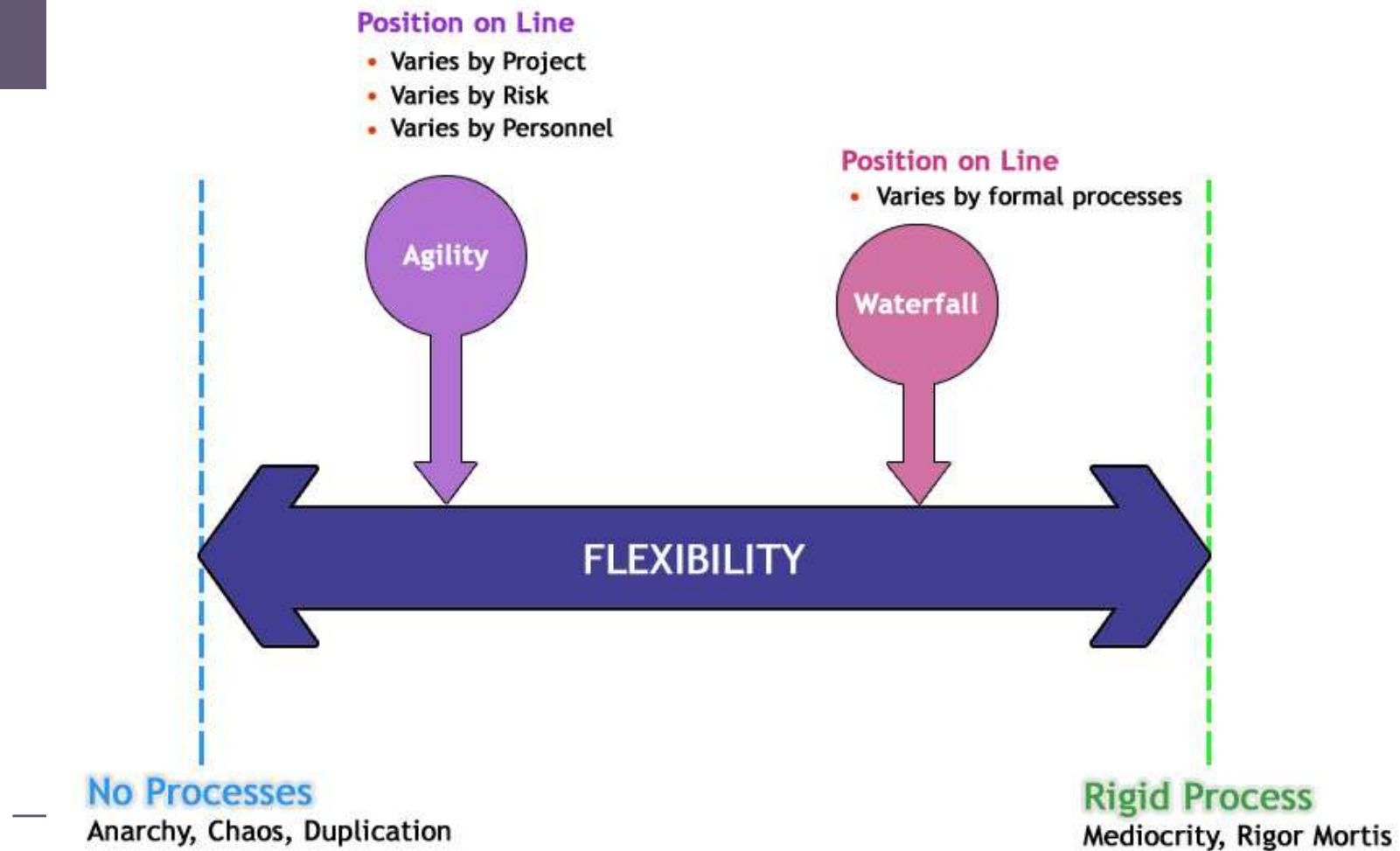
No. of Projects Success



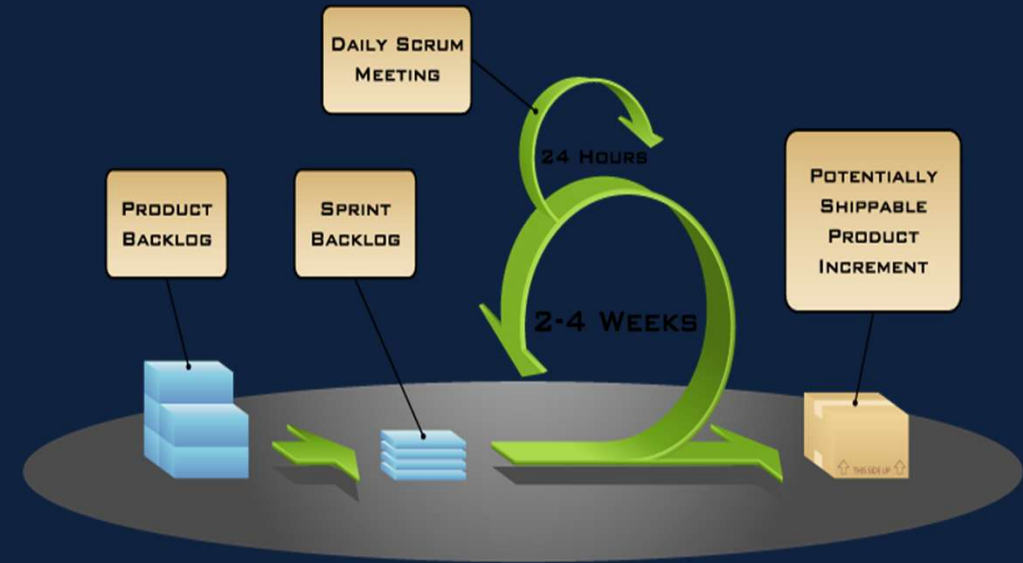


Source: The Standish Group

Adaptability Scale



Scrum



The world's most widely used agile framework —
60%+ market share

What is Scrum?

Definition

Scrum is a lightweight framework that helps teams deliver valuable products through short, repeatable cycles called sprints. It is not a process or a technique — it is a framework within which teams can use various processes and techniques.

Three things that make Scrum distinctive:



Time-boxed sprints

All work happens inside fixed 1–4 week cycles called sprints. The sprint end date never moves.



Inspect and adapt

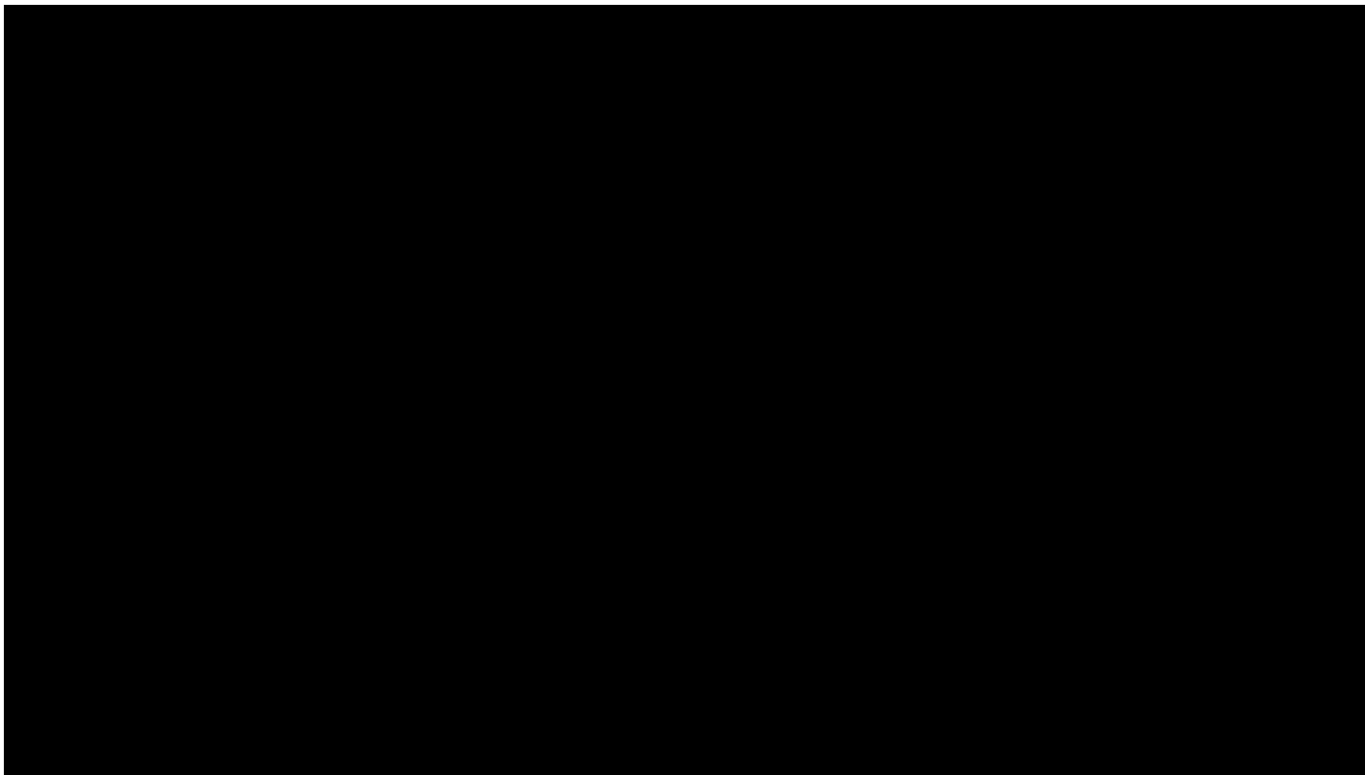
At every sprint boundary, the team examines what they built and improves how they work.



Empirical process control

Decisions are based on observation and experimentation — not a plan made months ago.

Scrum in 10 Minutes



Scrum — Five Core Values

The values are not aspirational — they are the foundation that makes the framework work

Commitment

Every team member commits personally to achieving the sprint goal

Focus

Everyone focuses on the sprint work — interruptions are minimised

Openness

The team and stakeholders are open about all work and all challenges

Respect

Team members treat each other as capable, independent professionals

Courage

Team members have the courage to do the right thing, even when hard

Scrum — Three Roles

These are not job titles — they are accountabilities within the Scrum team

Product Owner

Accountable for: What to build and in what order

- Manages and prioritises the Product Backlog
- Represents the needs of stakeholders and customers
- Accountable for maximising the value of the product
- Makes decisions about scope, priority, and release

Scrum Master

Accountable for: How the team works and improves

- Ensures Scrum is understood and followed correctly
- Facilitates all Scrum events at the right time
- Removes impediments blocking the team's progress
- Is a coach — not a project manager or line manager

Development Team

Accountable for: How to build the increment

- Cross-functional — has all skills to deliver a Sprint
- Self-organising — no one tells them how to do the work
- 3–9 members (optimal for coordination and speed)
- Jointly responsible for delivering the Sprint Goal

Scrum — Three Artifacts

Artifacts make work and progress transparent to everyone

Product Backlog

The ordered list of everything that might be needed in the product. The Product Owner manages it. It is never complete — it evolves as the product and market change.

Examples: user stories, bug fixes, features, improvements — all expressed as value to the customer

Sprint Backlog

The set of Product Backlog items selected for the current Sprint, plus the plan for delivering them and achieving the Sprint Goal. Owned by the Development Team.

The team creates it at Sprint Planning and updates it daily. Only the team can change it during the Sprint.

Product Increment

The sum of all completed Product Backlog items at the end of a Sprint. Must meet the Definition of Done and be potentially shippable — even if the team decides not to release.

Each Sprint adds to the increment. After 6 sprints, the increment includes everything built in all 6 sprints.

Scrum — Five Events

Every event has a maximum time-box — it ends when its purpose is achieved

1

Sprint

1–4 weeks

The container for all other events. A time-box within which the team delivers a potentially shippable increment.

2

Sprint Planning

Max 8 hours

The team collaborates to define the Sprint Goal and select the backlog items they will deliver in this Sprint.

3

Daily Scrum

15 minutes

A daily synchronisation meeting. Three questions: What did I do yesterday? What will I do today? Any blockers?

4

Sprint Review

Max 4 hours

The team demonstrates the increment to stakeholders and collects feedback to update the Product Backlog.

5

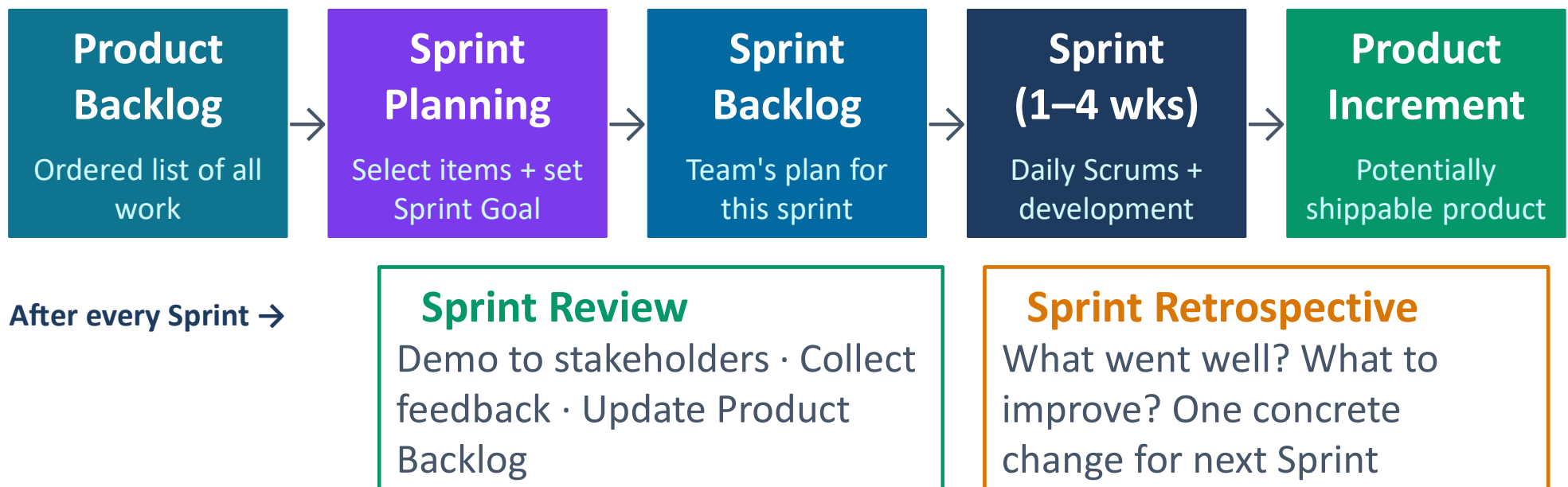
Sprint Retrospective

Max 3 hours

The team reflects on how they worked and identifies one or two concrete improvements for the next Sprint.

Scrum — The Sprint Cycle

Every sprint follows the same repeating pattern — inspect, adapt, deliver, repeat



*The output of Sprint Review feeds back into the Product Backlog
→ ready for the next Sprint Planning.*

Scrum — Definition of Done (DoD)

One of Scrum's most important quality concepts

What is it?

The Definition of Done is a shared understanding of what "complete" means. A Product Backlog item is not done until it meets every criterion in the DoD.

Why the Definition of Done matters:

- It creates transparency — everyone knows exactly what 'done' looks like
- It prevents undone work from accumulating as hidden technical debt
- Without it, different team members have different standards — leading to surprises at Sprint Review
- The DoD evolves over time as the team improves — stricter DoD = higher quality

*Example DoD: code written · unit tests pass · code reviewed · integrated into main ·
deployed to test environment · acceptance criteria verified*

Scrum — Strengths and Challenges

Strengths

- Delivers working software frequently — builds trust with stakeholders
- Continuous feedback reduces the risk of building the wrong thing
- Simple to understand; teams can be trained quickly
- Retrospectives create a culture of continuous improvement
- Works well for complex product development where requirements change

Challenges

- Requires cultural change — not just renaming roles and scheduling meetings
- Product Owner must be available and empowered — often a struggle in practice
- Scrum Master is frequently misunderstood as a project manager
- Does not scale naturally to very large or distributed teams
- Produces less documentation — can be a problem in regulated industries

Extreme Programming

Engineering excellence taken to the extreme — Kent Beck, 1999